Information Retrieval and Web Analytics

Final Project PART 4
Ranking and Filtering

Laura Naranjo & Laura Peñalver & Aurora Pujols

November 29, 2025

TAG: IRWA-2025-part-4
GitHub: https://github.com/aurorapujols/irwa-search-engine

We separate this part 4 of the project in three parts:

a. **Search Engine UI (1)** in which we explain our modifications and additions to the UI of the search engine.

b. **RAG(2)**: in which we detail what weaknesses we detected in the RAG and how we improved them.

c. **Web Analytics (3)**: in which we implement the mechanism for tracking and analyzing how users use the search engine.

IMPORTANT: we decided to implement the search engine in a way such that the search for the user takes only a few seconds. For that, all the expensive computations are performed at the beginning, when the search engine is started. Due to this, it takes some time for the engine to start and the progress can be seen in the console when starting the web application.

1 PART 1: Search Engine UI

AI use

CODE: because none of the group members had previous knowledge of HTML and CSS, we used ChatGPT as a tool for learning and helping us develop the code for the templates.

In the following list, we explain what we added to each part of the UI and how.

Search page: we simply changed the looks of the search page with the halp of AI, and also added the selecting tool for the user to choose which algorithm to rank the documents with (see Figure 1).

Search action: the text entered in the searching tab is passed to the `function`. To avoid problems, we added error control in case the user enters a blank query. In this case, the engine returns to the same page not computing any ranking. The following line shows the call to the search function:

```
results = search_engine.search(search_query, search_id, search_type=selected_model.lower
                                                    ())
```

Search function in the engine: as seen in the previous code, the search function receives additional parameters to the `search_query`. They are: the `search_id` to keep track in the UI of the search we are making, and the `search_type` wich is a string indicating the rank algorithm we need to use for that search. It can have four values: `['tf_idf', 'bm25', 'custom', 'word2vec']`.

Search algorithms: the previous `search` function will call, depending on the `search_type` one of the four ranking algorithms as shown in the code below. The `get_(...)_ranking` functions are defined in the `algorithms.py` file, and call the defined functions in part 3 of the project to get the rankings of the products.

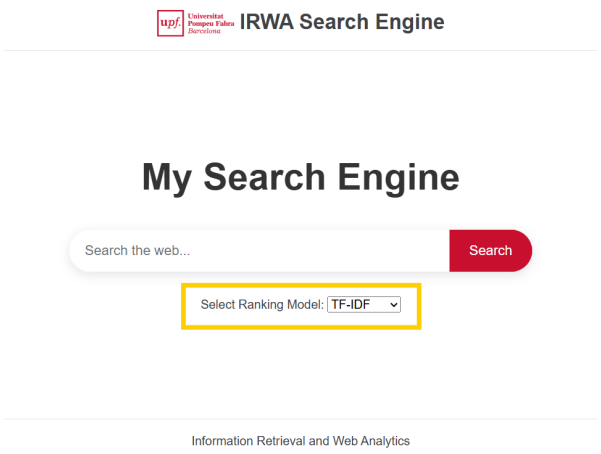


Figure 1: Search Page of the engine. In yellow, the additional selection tool we added for the ranking algorithm.

```
def search(self, search_query, search_id, search_type='tf-idf'):
    print("Search query:", search_query)

    rankings = []

    # WITH FILTERING:
    # query_terms, filtered_prod = filter(search_query, self.products_dict)

    # WITHOUT FILTERING:
    query_terms = build_terms(search_query)
    filtered_prod = list(self.products_dict.values())     # all of the products

    results = []
    ### You should implement your search logic here:
    # results = dummy_search(corpus, search_id)
    if(query_terms and filtered_prod):
        match search_type:
            case 'tf-idf':
                rankings = get_tf_idf_ranking(query_terms, filtered_prod, self.
                                                tf_idf_index, self.tf_tfidf
                                                , self.idf_tfidf, self.
                                                weights_tf_idf)
            case 'bm25':
                rankings = get_bm25_ranking(query_terms, self.products_dict,
                                              filtered_prod, self.idf_bm25
                                              , self.Ld, self.Lave)
            case 'custom':
                rankings = get_custom_ranking(query_terms, self.metadata, self.
                                              products_dict, filtered_prod
                                              , self.idf_bm25, self.Ld,
                                              self.Lave)
            case 'word2vec':
                rankings = get_word2vec_ranking(search_query, self.word2vec_model, self.
                                              prod_embeddings,
                                              filtered_prod, preprocess=
                                              preprocess_string)

        results = get_search_results(corpus=self.corpus, results=rankings, search_id=
                                     search_id)
    # results = search_in_corpus(search_query)
    return results
```

The `get_search_results` function transforms the result given by the algorithms into the correct format for the search engine to display them on the web. We can also see how in the comment `# WITH FILTERING` we first implemented the search engine with the filtering used in the last part. However, we later added all the products in the corpus to the search.

The results page: the results page was modified to include the asked fields of each document into the preview of each product (see Figure 2).

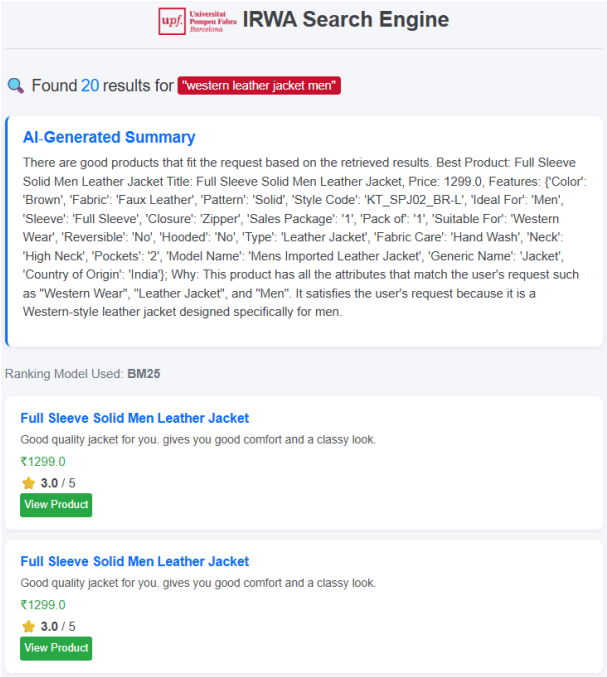


Figure 2: Results page of the search engine, including an LLM generated recommendation at the top.

LLM summary on the resutls page: we used the already provided code to implement this part.

Document details page: we also modified the UI in this page to get all the values of each product and show it in a visually appealing way (see Figure 3).

2 PART 2: RAG

The RAG we are using has a specified template prompt that produces an answer to the question of making a recommendation of the products in the retrieved products list. However, we saw 3 main weaknesses that we improved:

2.1 Fields given to the LLM

We observed how the LLM did not show base its answer using all the attributes we have for a given product. Only the `pid` and `title` of the products were given as input in the prompt. This meant that the LLM based its answer purely on these fields.

We solved it by changing the part of the code that provides the information of the products in the `generate_response` function:

```
formatted_results = "\n".join([ f"""- PID: {res.pid}, Title: {res.title}, Price: {res.
                                    selling_price}, Rating: {res.average_rating}
                                    , Category: {res.category}, Brand: {res.
                                    brand}, Description: {res.description},
                                    Features: {res.product_details}"""
                                for res in retrieved_results[:top_N]])
```

We added as information to analyze the price, the rating, the category, the brand, the description, and the details of the product. In figure 4 we can see how the answer including more fields references to them to have more information for the recommendation.

See the pros and cons of this improvement in table 1.

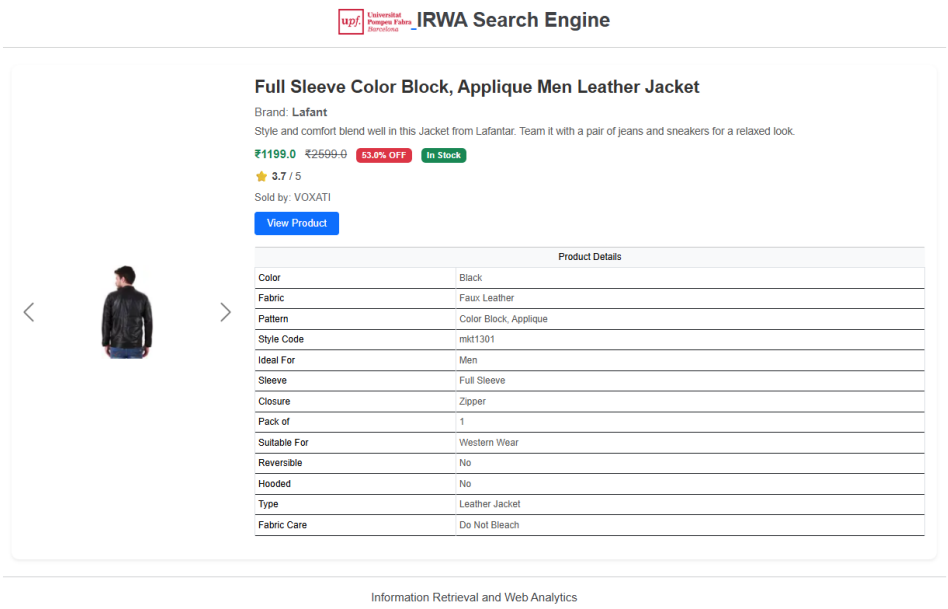
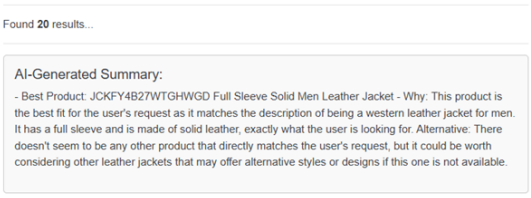
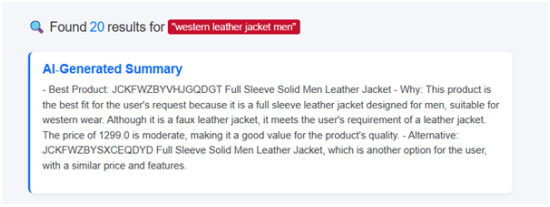


Figure 3: Document details page for a specific product.



(a) Original RAG answer.



(b) RAG answer when it bases the recommendation on more fields.

Figure 4: RAG answers before and after adding the extra fields. (a) bases all recommendations on what the title says, and (b) bases the answer also in other fields of the product.

PROS	CONS
Gives richer context allowing more accurate recommendations. The reason and explainability of the LLM improve.	Increases prompt length and affects cost and latency for API calls. Risk of information overload (not focus on important fields). Data leakage if the data is private.

Table 1: Pros and cons of adding more fields in the LLM prompt.

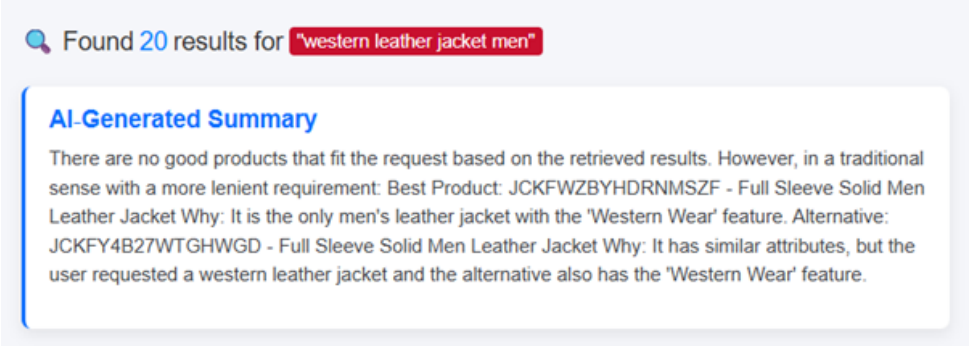


Figure 5: Result of RAG after improving the prompt.

2.2 Prompt Engineering

The current prompt is vary basic, so we decided to extend it and be more precise on what we wanted the LLM to do and prompt out.

In table 2 there are the pros and cons of adding this improvement.

PROS	CONS
Produce more relevant, task-specific responses. Reduces irrelevant or vague answers. Improves consistency and reliability.	LLM might overfit to prompt wording and fail on some queries. Making a long prompt can influence the time the answer is given in.

Table 2: Pros and cons of improving the prompt.

2.3 Hallucination

We can see how the LLM will always provide an answer to the recommendation. But it could be the case in which none of the ranked products is a good recommendation for the user. In those cases, the LLM hallucinates and provides a not good recommendation. What we did was improve the prompt so that the LLM is aware of this and explicitly says that it did not find any good recommendation.

In Figure 6 we can see a version in which the LLM finds a good recommendation and one in which it does not and says it.

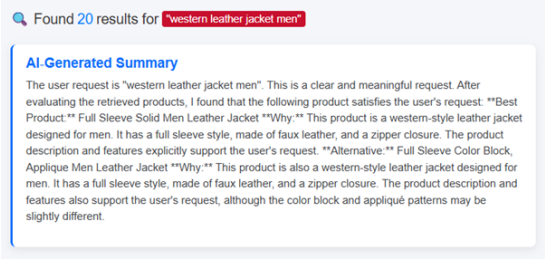
IMPORTANT: notice that the LLM is not deterministic, for the same query and results it might have slightly different answers each time. Because of this, we realized that in some cases it answers that it did not find any good recommendation but tries to explain which product would be best anyway out of all the retrieved ones (see Figure 5).
IMPORTANT: we have noticed that after improving the prompt, the ranking after the search button is pressed increases by a few seconds. This shows how the improvements in the prompt are, indeed, detrimental to the performance time.

The final prompt we use is the following:

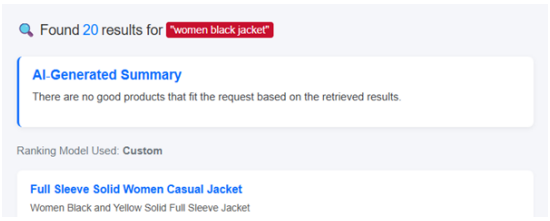
```
PROMPT_TEMPLATE = """
```

PROS	CONS
Reduces misleading recommendations. More trustworthy system. Improves user experience.	It is non-deterministic so it may still attempt to recommedn when not needed. Might produce less helpfull answers. Reduces the response time further.

Table 3: Pros and cons of avoiding hallucinations.



(a) RAG answer when it found a good product to recommend.



(b) RAG answer when it did not find a good product to recommend.

Figure 6: RAG answers after all the improvements. (a) example when it finds a recommendation, and (b) example when it does not find a good recommendation.

```
        You are an expert product advisor for an e-commerce system. You will receive
                                                retrieved products with structured
                                                metadata.

        1. Evaluate if any retrieved product meaningfully satisfies the user's request:
        - A product satisfies the request ONLY if its attributes (price, features,
                                                category, specs, etc.) explicitly
                                                support the request.
        - Cite only fields present in the retrieved products as evidence.
        - If none qualify, respond ONLY AND EXACTLY: "There are no good products that fit
                                                 the request based on the retrieved
                                                results."
        - If the user query is empty or nonsensical, respond ONLY AND EXACTLY: "The user
                                                request is empty or does not make
                                                sense."
        - Do NOT infer or guess missing information.

        2. Present the recommendation clearly in this format using plain text and maximum
                                                 150 words:
        - Best Product: Title
        - Why: Explanation in plain language why it is the best fit, referring only
                                                retrieved product fields if needed
        - Alternative (optional): ONE alternative, with its attributes

        IMPORTANT: Do not mention anything not in the retrieved metadata.

        ## Retrieved Products:
        {retrieved_results}

        ## User Request:
        {user_query}

    """
```

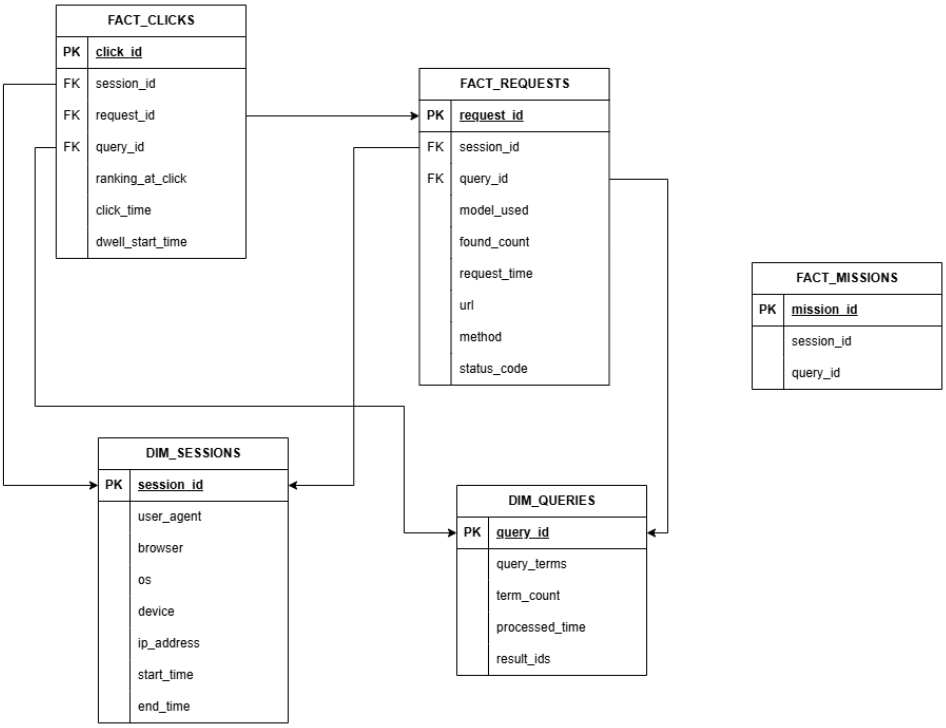


Figure 7: Schema of the data storage.

3 PART 3: Web Analytics

AI use

CODE: because none of the group members had previous knowledge of HTML and CSS, we used ChatGPT as a tool for learning and helping us develop the code for the dashboard template as well as the plots and how to display them in the dashboard.
THEORY: because it was hard to understand what we needed to do with all the already existing files and what we were being asked to do, we used (mainly Google) but also sometimes AI tools to help us figure out the way the schema we had planned would fit in the application.

3.1 Data Collection & Storage

We started by designing the star schema of our website to know which data, how and where to store it. The schema of how we organized the data can be seen in Figure 7.

We are additionally storing some variables other than the ones mentioned in the lab instructions because they helpped us compute the statistics and ranking and some (like `dwell_time`) the variables themselves. We also added some data that we ended up not using as we wanted and might be in the design but not the code itself. And some parts of code that started being implemented but not finished due to the lack of time like the `fact_missions`.

Then, all of the tables are stored in `DataAnalytics` as dictionaries. And they are filled as it needs in the file `web_app.py`. In which when the user enters the page, initiates a session, when it does a request and goes look at the search results it starts a query, and the fact tables of clicks and requests store relevant information to compute the dashboard aftwerwards.

Some details to take into account:

a. The `save_(...)` functions are used to store the analytics data into the tables.

b. The `get_(...)` functions are the ones that read the tables to compute the rankings, and statistics.

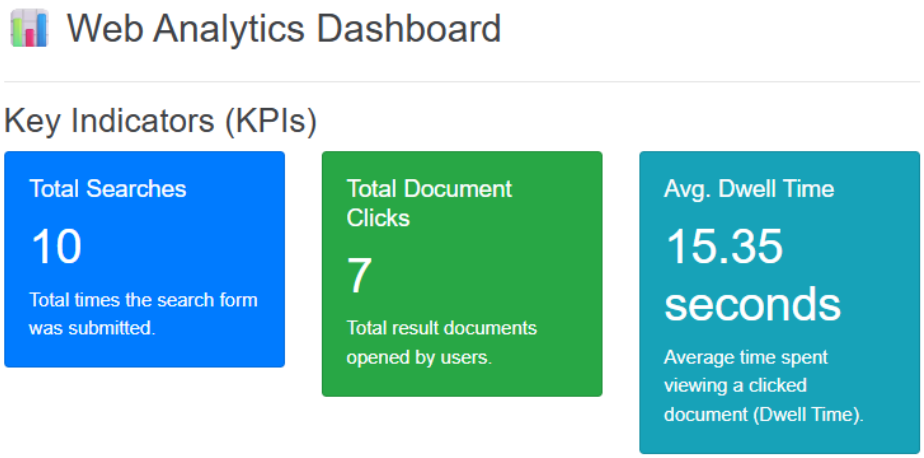


Figure 8: Dashboard KPIs.

c. The `plot_(...)` functions are the ones that help us create the plots that will be displayed in the dashboard.

The routes in `web_app.py` are the ones that control the movement of the user as well as the clicks. From there we access the `DataAnalytics` data and, with it, the other tables. It also has the code to check whether or not we have enough statistics to display the dashboard.

3.2 Analytics Dashboard

In this part, we show what our dashboard looks like while explaining the results and how the users use the website based on the images.

The first thing that is displayed is the **KPIs** (see Figure 8. In them we can see the **total number of searches** made in the engine, the **total number of clicked documents**, and the **average dwell time**. We can see how in the screenshot, the engine does not have a lot of clicked documents (less than searches) and that the users did not spend much time on the documents (average of 15s), only too look like an overview.

Then we can see the two graphs in Figure 9. The first one is better seen in Figure 10 and shows the top most clicked documents which in this case are two: "Men Regular Fit..." and "Women Solid Khadi...". The engines (not seen in the image) are Google chrome in blue, and Microsoft Edge in orange.

In Figure 11 we can see the distribution of term counts (the number of terms each query has). We can see how only queries of 3 and 4 terms have been made so far with more of 3 than 4.

Finally, in Figure 12 we can see three tables. The first one (top left) has the top terms in the queries. We can see how the most used word in the queries is "men". The second table (top right) has the TOP visitors' IP and we can see how only one user has used the website. The third and last table (bottom), has the top clicked documents (how many times they were clicked) and their ranking in the search results.

4 Comments

We wanted to additionally say, that even though we learned a lot while doing this practice, we find that there was few information in the theory lessons that could help us develop this final part of the project. And also, even thought the deadline was extended we were still a bit short on time.

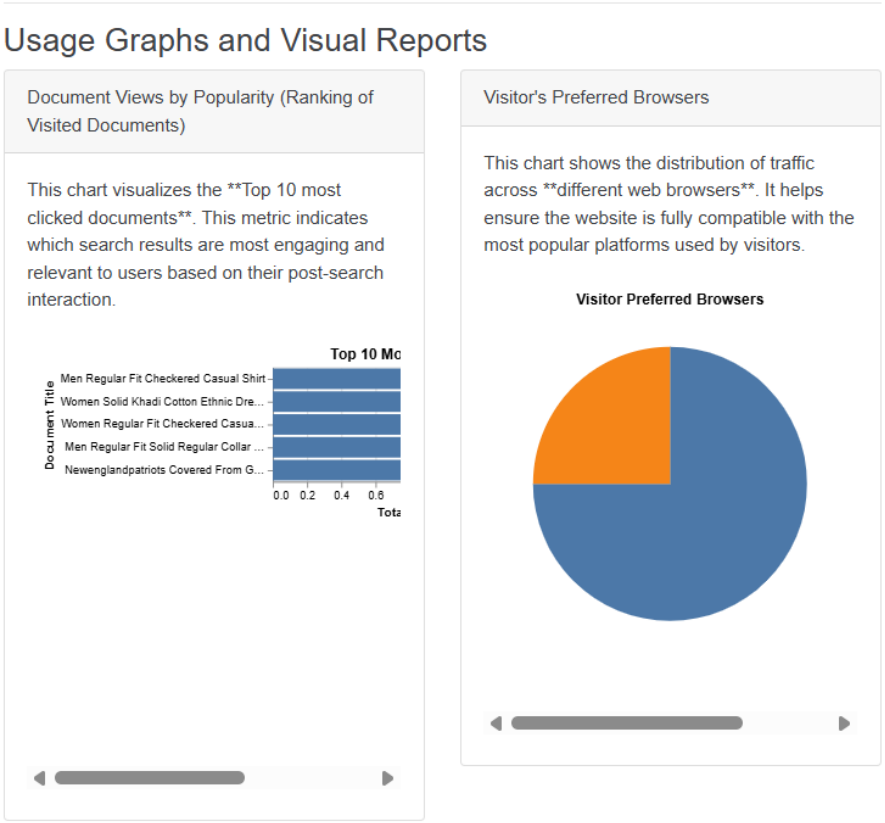


Figure 9: Top documents (better viewed in next figure) and percentage of engines used.

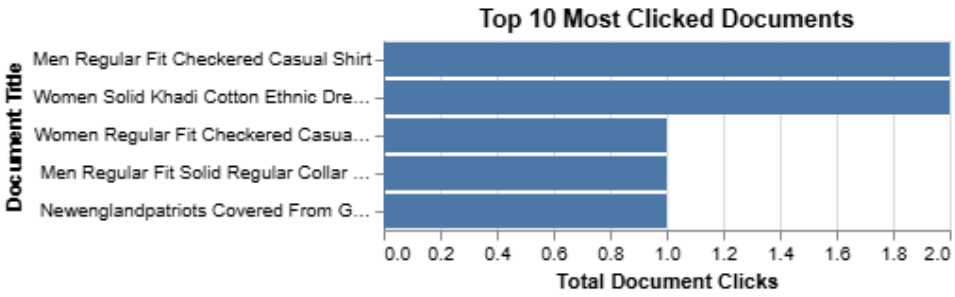


Figure 10: Top documents.

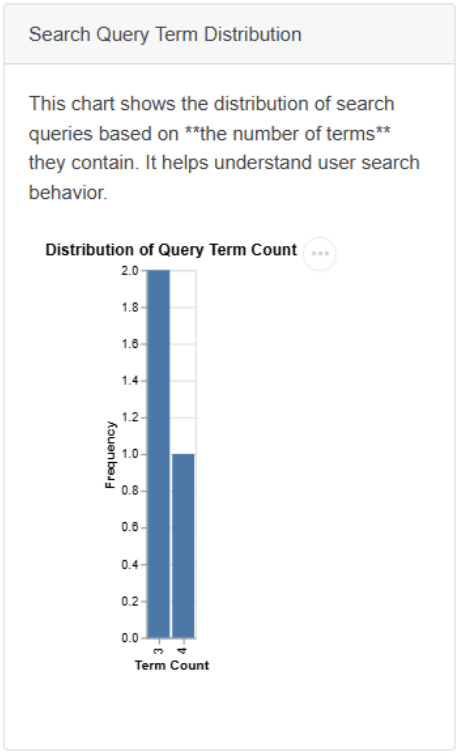


Figure 11: Query term count distribution.

Preferred Terms

The top 10 most common single words (terms) used in all searches.

Rank	Term	Count
1	men	**2**
2	western	**1**
3	leather	**1**
4	jacket	**1**
5	women	**1**
6	purple	**1**
7	dress	**1**
8	weather	**1**
9	raincoat	**1**

Visitor's City (Top IPs)

Tracking visitor location (using IP address as a proxy for city/country - Bonus Point). To get city names, a GeoIP library is required.

Rank	IP Address	Session Count
1	127.0.0.1	**16**

Detailed Click Ranking Report

A detailed view of the **Ranking of visited documents** by click count.

Rank	Product ID (PID)	Description (Title)	Total Clicks
1	SHTFRTGXFZFH4PJE	Men Regular Fit Checkered Casual Shirt	**2**
2	SHTFSH5VRDYPSGGK	Women Regular Fit Checkered Casual Shirt	**1**
3	SHTEMD8RTVX3UWJB	Men Regular Fit Solid Regular Collar Formal Shirt	**1**
4	KTAFV92XN8VJK8AY	Women Solid Khadi Cotton Ethnic Dress (Purple)	**1**

Figure 12: Tables with top data.